7. NOMBRES PREMIERS

7.1. DEFINITIONS ET PROPRIETES

DEFINITION

Un nombre p est premier s'il admet exactement deux diviseurs, 1 et lui-même
Remarque : de fait, l'entier 1 n'est pas premier, puisqu'il n'admet qu'un seul diviseur.

PROPRIETE

- L'ensemble des nombres premiers est infini.
- Tout entier différent de 0 et 1 peut se décomposer en produit de facteurs premiers :

$$n = p_1^{k_1} \times p_2^{k_2} \times \ldots \times p_m^{k_m}$$

Démonstration par l'absurde de la première propriété
Supposons que l'ensemble P des nombres premiers soit fini et égal à $\{p_1, p_2, \ldots, p_n\}$.
Prenons l'entier N $= \mathbf{p_1} \times \boldsymbol{p_2} \times \ldots \times \boldsymbol{p_n} + \mathbf{1}$ **…** Terminez le raisonnement…

EXERCICE 13

1. Décomposer en produit de facteurs premiers les entiers 60, 972,1500
2. On considère l'entier $n = 120$.
   - Décomposer 120 en produit de facteurs premiers.
   - Déterminer le nombre de diviseurs de 120.
3. Généralisation : on considère l'entier $n = p_1^{k_1} \times p_2^{k_2} \times \ldots \times p_m^{k_m}$.
   Déterminer le nombre de diviseurs de n.

COMPLEMENTS FACULTATIFS

EXERCICE 14

Soit n un entier non nul, sa **factorielle**, notée $\boldsymbol{n}$! est définie par : $n! = 1 \times 2 \times \ldots \times n$

1. Décomposer 6 ! et 9 ! en produit de facteurs premiers.
2. Décomposer $(6!)^2$ et $(9!)^3$ en produit de facteurs premiers.

EXERCICE 15

1. Expliquer pourquoi les nombres suivants $2^3 \times 3^4 \times 5^2$ et $5^3 \times 7^2 \times 11^4$ ont le même nombre de diviseurs.
2. Un entier n a exactement 80 diviseurs. Quel est ce nombre sachant que sa décomposition en facteurs premiers comporte uniquement 2 élevé à la puissance 4 ainsi que 3 et 5 élevés à la même puissance.
3. Un entier n a exactement 27 diviseurs. Quel est ce nombre sachant que sa décomposition en facteurs premiers comporte uniquement 2, 3 et 5 élevés à la même puissance.

EXERCICE 16

Ecrire un algorithme, puis un programme Python permettant de créer une fonction **premier(n)** prenant en entrée un entier naturel **n** et retournant un booléen : **True** si l'entier n est premier et **False** s'il ne l'est pas.
Faire en sorte que la complexité (ici, le nombre d'étapes dans la boucle utilisée) soit la plus faible possible.
Deux opérateurs Python importants :

- a // b : donne le **quotient** de la division euclidienne de a par b
- a % b : donne le **reste** de la division euclidienne de a par b

VALUATIONS P-ADIQUES

Si nous notons P l'ensemble des nombres premiers et n un entier naturel quelconque, nous écrirons sa décomposition en facteurs premiers sous la forme :

$$n = 2^{v_2} \times 3^{v_3} \times 5^{v_5} \times 7^{v_7} \times 11^{v_{11}} \times 13^{v_{13}} \times 17^{v_{17}} \times \ldots \times p^{v_p} \times \ldots$$

Ce produit est infini, car il y a un nombre infini de nombres premiers, cependant il n'y a qu'un nombre fini de v_p non nuls et donc un nombre infini de termes p^{v_p} égaux à 1 ($p^{v_p} = p^0 = 1$)
Pour écrire ce produit nous pouvons utilise un « grand opérateur » $\prod$ qui a l'instar de la somme Σ permet d'écrire le produit d'une suite de nombres :

$$n = \prod_{p \in P} p^{v_p}$$

Dans cette décomposition, v_p dépend de n et il serait plus correct de le noter $v_p(n)$.

Exemple : $882 = 2 \times 3^2 \times 7^2$
$\Rightarrow \boldsymbol{v_2(882) = 1}$ **;** $\boldsymbol{v_3(882) = 2}$ **;** $\boldsymbol{v_5(882) = 0}$; $\boldsymbol{v_7(882) = 2}$; $\boldsymbol{v_{11}(882) = v_{13}(882) = 0}$ …

$v_p\ (n)$ est appelée valuation p-adique de n

4. PGCD – ALGORITHME D'EUCLIDE

4.1. DEFINITION

Le **pgcd** de deux entiers relatifs a et b est le « **plus grand commun diviseur de a et b** », c'est-à-dire le plus grand entier d divisant à la fois a et b.

EXERCICE 17
Déterminer : $\text{pgcd}(12,20)$, $\text{pgcd}(72,60)$, $\text{pgcd}(54,45)$

EXERCICE 18
Montrer que si $a = bq + r$ alors $\text{pgcd}(a,b) = \text{pgcd}(b,r)$

COMPLEMENTS FACULTATIFS
EXERCICE 19
Montrer qu'il n'existe pas d'entiers m et n tels que $m + n = 101$ et $\text{pgcd}(m,n) = 3$

EXERCICE 20
Trouver tous les entiers positifs a et b, avec $a < b$, tels que :
- $\text{pgcd}(a,b) = 4$
- $ab = 192$

EXERCICE 21
Trouver tous les entiers positifs a et b, avec $a < b$, tels que :
- $\text{pgcd}(a,b) = 9$
- $a + b = 72$

4.2. METHODES DE CALCUL DU PGCD

4.2.1. EN UTILISANT LA DECOMPOSITION EN FACTEURS PREMIERS

Prenons par exemple 540 et 6174.
Nous décomposons les deux entiers en facteurs premiers : $540 = 2^2 \times 3^3 \times 5$ et $6174 = 2 \times 3^2 \times 7^3$
Il suffit de reprendre les éléments de cette décomposition :

- 2^2 divise 540 et $2 = 2^1$ divise 6174 : la plus grande puissance de 2 qui divise les deux nombres est donc $\mathbf{2} = 2^1$
- De même, la plus grande puissance de 3 divisant 540 et 6174 est $\mathbf{3^2}$
- 5 ne divise pas 882, il n'est pas diviseur commun à 540 et 6174. On peut aussi dire que la plus grande puissance de 5 divisant les deux entiers est 5^0 (qui divise 6174)
- Idem pour 7

Bilan : $\text{pgcd}(540,6174) = 2 \times 3^2 = 18$

RAPPEL …
La décomposition en facteurs premiers d'un entier n quelconque peut s'écrire de la façon suivante :

$$n = \prod_{p \in P} p^{v_p(n)}$$

Dans cette décomposition, $v_p(n)$ est la valuation p-adique de n
Nous pouvons donc en déduire une nouvelle définition du pgcd :

$$\text{pgcd}(n,m) = \prod_{p \in P} p^{min(v_p(n),v_p(m))}$$

REMARQUE : nous pourrions chercher à programmer une fonction $\text{pgcd}(n,m)$ en utilisant la formule ci-dessus et le programme de décomposition en facteur premier. Mais sur des grands nombres, cette méthode est trop lente et rend pratiquement impossible leur décomposition.
Nous allons voir une autre méthode algorithmiquement plus intéressante.

4.2.2. EN UTILISANT LA DIVISION EUCLIDIENNE ET LES PROPRIETES DU PGCD

EXEMPLE 1 (REPRISE DE L'EXERCICE 18)

Rappelons la propriété :

si c divise a et b alors c divise toute **combinaison linéaire** de a et b :

$$\left.\begin{matrix} c|a \\ c|b \end{matrix}\right\} \Rightarrow \boldsymbol{c|\lambda a + \mu b}, \quad \forall \boldsymbol{\lambda, \mu} \in \boldsymbol{Z}$$

Considérons deux entiers n et m $(n \geq m)$ et soit q et r, respectivement le quotient et le reste de la division euclidienne de n par m : $n = mq + r$

Soit $d = pgcd(n, m)$:

$$\left.\begin{matrix} d|n \\ d|m \end{matrix}\right\} \Rightarrow d|\ (n - mq) \Rightarrow\ d\ |r\ \Rightarrow d|\ \mathrm{pgcd}(r, m)$$

On a donc $d|\ d'\ avec\ d' = \mathrm{pgcd}(r, m)$

Réciproquement : on peut montrer que d' divise

$$\left.\begin{matrix} d'|r \\ d'|m \end{matrix}\right\} \Rightarrow d|\ mq + r \Rightarrow\ d\ |n\ \Rightarrow d|\ \mathrm{pgcd}(n, m) \Rightarrow d|d'$$

Conclusion : $d\ =\ d' \Rightarrow \boldsymbol{\mathrm{pgcd}(n, m) = \mathrm{pgcd}(r, m)}$

Propriété du PGCD : si q et r sont respectivement le quotient et le reste de la division euclidienne de n par m $(n \geq m)$ alors : $\boldsymbol{\mathrm{pgcd}(n, m) = \mathrm{pgcd}(m, r)}$

En raisonnant ainsi :

$\boldsymbol{6174 = 540 \times 11 + 234}$ (division euclidienne de 6174 par 540) $\Rightarrow \boldsymbol{\mathrm{pgcd}(6174, 540) = \mathrm{pgcd}(540, 234)}$

$\boldsymbol{540 = 234 \times 2 + 72}$ (division euclidienne de 540 par 234) $\Rightarrow \boldsymbol{\mathrm{pgcd}(540, 234) = \mathrm{pgcd}(234, 72)}$

$\boldsymbol{234 = 72 \times 3 + 18}$ (division euclidienne de 234 par 72) $\Rightarrow \boldsymbol{\mathrm{pgcd}(540, 234) = \mathrm{pgcd}(72, 18)}$

$\boldsymbol{72 = 18 \times 4 + 0}$ (division euclidienne de 72 par 18) $\Rightarrow 18\ divise\ 72 \Rightarrow \boldsymbol{\mathrm{pgcd}(72, 18) = 18}$

CONCLUSION : $\boldsymbol{\mathrm{pgcd}(6174, 540) = 18}$

BILAN : ALGORITHME D'EUCLIDE

L'algorithme d'Euclide est une méthode itérative permettant de trouver le **plus grand commun diviseur (PGCD)** de deux nombres entiers.

PRINCIPE

Le PGCD de deux nombres n par m $(n \geq m)$ est égal au PGCD de m et du reste r de la division de n par m.

ÉTAPES DE L'ALGORITHME

1. Diviser le plus grand nombre par le plus petit.
2. Remplacer le plus grand nombre par le plus petit, et le plus petit par le reste de la division.
3. Répéter jusqu'à ce que le reste soit 0.
4. Le PGCD est le dernier reste non nul.

EXEMPLE 2

Cherchons le **pgcd** de 9900 et 594

La division de	Par	Donne le reste
9900	594	396
594	396	198
396	198	0

Le PGCD de 9900 et 594 est donc **198**

EXERCICE 22
En utilisant les deux méthodes précédentes (décomposition en facteurs premiers et algorithme d'Euclide), déterminer :

- pgcd(720,48)
- pgcd(241,42)

EXERCICE 23 – POUR LES PLUS AVANCES
Ecrire un algorithme et un programme Python permettant de créer une fonction **euclide(n,m),** prenant en paramètres deux entiers **m** et **n** et retournant leur **pgcd**.

4.2.3. COMPLEXITE DES DEUX METHODES

Il est encore un peu tôt pour aborder la notion de complexité algorithmique de manière rigoureuse.
En cours de mathématiques, nous étudierons la complexité en grand O, qui consiste à estimer le nombre d'itérations de l'algorithme, dans le pire des cas. Nous en donnerons une **définition plus formelle** à la fin du deuxième semestre.

Concernant les méthodes précédentes — **l'algorithme d'Euclide** pour le calcul du PGCD et les **méthodes de décomposition en facteurs premiers** — leur complexité n'est pas simple à établir précisément.
Si l'on exprime cette complexité en fonction du **nombre** de bits b utilisés pour représenter les entiers m et n, on obtient :

- Pour l'algorithme d'Euclide, une complexité **majorée par** $O(b^3)$.
- Pour la décomposition en facteurs premiers, une complexité en $O(\mathrm{e}^{1.923(b\ln 2)^{1/3}(\ln(b\ln 2))^{2/3}})$

Pour les théoriciens de la complexité, il y a une différence fondamentale entre ces deux complexités.

- la première complexité est **polynomiale** (b^3, polynôme de degré 3), ce qui est considéré comme « efficace ».
- la seconde est **supra-polynomiale** — elle croît plus vite que tout polynôme, sans être ici tout à fait exponentielle. En pratique, cela signifie que les algorithmes de décomposition deviennent très lents pour de grandes valeurs de b.
  Cette difficulté à décomposer de très grands entiers est utilisée pour sécuriser les communications, nous le verrons en semestre 3 avec **le chiffrement et la signature RSA**. A la base de cette méthode se trouve la factorisation de nombres RSA, qui sont des entiers obtenus par produit de deux nombres premiers.

Illustrons numériquement ces résultats :

Estimations de la complexité

Bits	**Chiffres décimaux**	**Euclide $O(b^3)$ (1 ns par opération)**	**Décomposition en facteurs premiers $O(e^{1.923(bln2)^{1/3}(ln(bln2))^{2/3}})$ (1 ns par opération)**	**Exemples historiques de factorisation de nombres RSA**
128	39	≈ 2 ms	≈ 13 s	
256	77	≈ 16 ms	≈ 27 h	
512	155	≈ 130 ms	≈ 350 ans	
768	232	≈ 0,5 s	≈ 10^6 ans	≈ 2000 années-cœur CPU Collaboration internationale, environ 2 ans en calculs parallèles - 2009
829	250	≈ 0,7 s	≈ 10^9 ans	≈ 2700 années-cœur CPU Collaboration internationale, environ 10000 ordinateurs sur quelques mois - 2020
1024	309	≈ 1 s	≈ 10^{10} ans	Jamais factorisé
2048	617	≈ 4 s	≈ 2.4×10^{19} ans	Jamais factorisé
4096	1234	≈ 16 s	≈ 1.6×10^{33} ans	Jamais factorisé